TRADINGLAB 2.0 ? DOCUMENTO DE ARQUITECTURA

TradingLab 2.0 — Pipeline SQX → MT5 → Incubación

Proceso controlado de preparación, compilación, despliegue e incubación

Versión: 1.1
Estado: Propuesta formal incorporada a Arquitectura 2.2
Fecha: 7 de octubre de 2026
Documento relacionado: `tradinglab-architecture-v2.md`

1. Propósito

Este documento define el proceso controlado para transformar una estrategia candidata generada en StrategyQuant X (SQX) en un bot compilado, validado, desplegado e incubado en MetaTrader 5 (MT5).

El objetivo es evitar que archivos exportados o ejecutables sin identidad verificable lleguen a incubación. Cada bot ejecutado debe poder relacionarse con su estrategia, exportación original, preparación, compilador, configuración, entorno y resultados. La VPS MT5 es una zona externa sin acceso remoto desde TradingLab; el intercambio inicial se realiza mediante el TradingLab MT5 Runner y paquetes transferidos por Google Drive.

2. Alcance

El pipeline comienza cuando una estrategia de SQX es seleccionada como candidata y termina cuando su incubación se cierra con una decisión registrada.

Incluye:

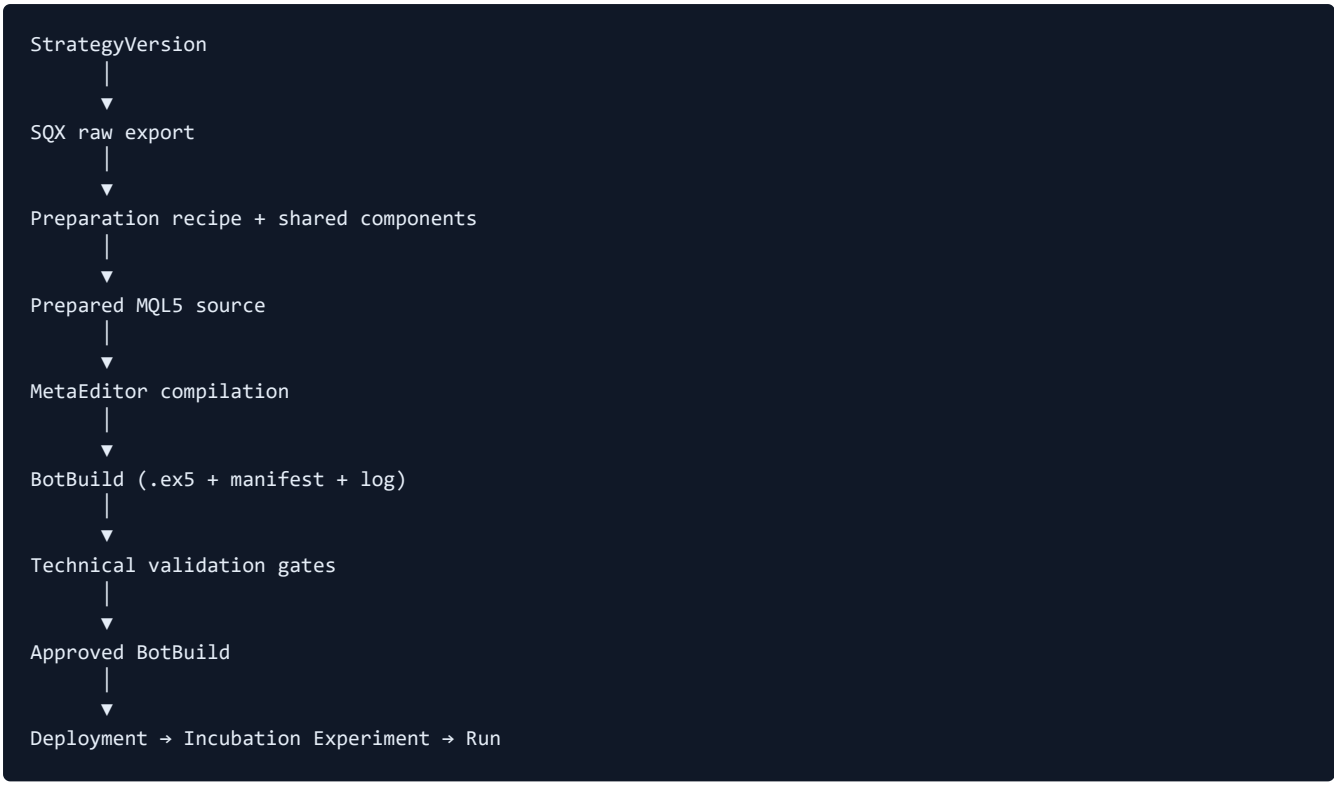
- selección y congelación de la estrategia;
- exportación y conservación del original de SQX;
- preparación técnica y funcional para MT5;
- compilación reproducible;
- pruebas y gates de calidad;
- empaquetado y aprobación;
- despliegue en un entorno MT5;
- incubación, observabilidad y cierre.

No incluye todavía promoción a trading real. Esa transición requerirá una política y aprobación independientes.

El workflow operativo del runner se define en `tradinglab-mt5-runner-workflow.md`.

3. Principio central

Nunca se desplegará "el último `.ex5`" o "el archivo que está en la carpeta". Se desplegará un `BotBuild` aprobado, identificado por su ID y por el hash de su ejecutable.



Entre el build y la ejecución existe una frontera de confianza:



Drive transporta copias; storage y catálogo conservan los registros autoritativos.

4. Conceptos

`StrategyVersion`

Versión inmutable de la lógica y configuración de una estrategia. Es el origen conceptual del bot.

Exportación original SQX

Archivo producido directamente por SQX antes de cualquier adaptación. Se conserva como evidencia inmutable en storage.

Receta de preparación

Definición versionada de los cambios necesarios para convertir la exportación en código apto para TradingLab y MT5. Puede combinar configuración, scripts y componentes compartidos.

Código MQL5 preparado

Código resultante de aplicar la receta. Debe ser determinista siempre que sea posible. Si hay cambios manuales, deben registrarse mediante Git y revisión.

`BotBuild`

Construcción concreta de un bot: entradas, compilador, outputs, logs, hashes y estado de aprobación.

`Environment`

Descripción del contexto donde se ejecutará el bot: terminal, broker, cuenta mediante alias, símbolos, zona horaria y características relevantes.

`Deployment`

Registro que vincula un `BotBuild` aprobado con un `Environment` y una configuración efectiva.

Incubación

Experimento de observación controlada del bot en MT5. Puede ejecutarse en demo u otro entorno autorizado. Su resultado no cambia retrospectivamente el build.

5. Estados

Estados de BotBuild



- `draft` : registro creado, entradas todavía incompletas.
- `prepared` : código preparado y hashes calculados.
- `compiling` : compilación en curso.
- `compiled` : compilación correcta; aún no aprobado.
- `failed` : error de preparación o compilación.
- `tested` : pruebas técnicas completadas.
- `approved` : autorizado para incubación.
- `rejected` : no cumple los gates.
- `superseded` : existe otro build que lo reemplaza; conserva su historia.

Estados de Deployment

```
planned → deployed → active → stopped
             |          |
             └→ failed  └→ rolled_back
```

6. Stage 0 — Selección del candidato

Entrada

Una estrategia SQX que superó los criterios de selección del proyecto.

Procedimiento

1. Confirmar proyecto y estrategia.
2. Registrar o seleccionar una `StrategyVersion`.
3. Documentar el motivo de selección.
4. Registrar dataset, periodo, criterios IS/OOS y versión de SQX.
5. Confirmar que los resultados de origen y configuración están disponibles.

Gate G0

- Identidad de estrategia inequívoca.
- Configuración SQX conservada.
- Resultados que justifican la selección disponibles.
- Dataset y versión de SQX conocidos.

7. Stage 1 — Exportación y preservación

Procedimiento

1. Exportar utilizando un perfil de exportación identificado.
2. Copiar el archivo a un área de ingreso temporal.
3. Calcular SHA-256 antes de modificarlo.
4. Registrar media type, tamaño, fecha y versión de SQX.
5. Almacenar el original como artefacto inmutable con rol `sqx-raw-export`.
6. Asociarlo a la `StrategyVersion` y al futuro `BotBuild`.

Gate G1

- Hash registrado y verificable.
- Original preservado.
- Perfil y build de SQX identificados.
- Ninguna modificación previa a la captura.

8. Stage 2 — Preparación para MT5

Tipos de adaptación

- Normalización de nombre y metadatos.
- Asignación de Magic Number único.
- Mapeo de símbolos y sufijos del broker.
- Revisión de timeframe.
- Parámetros de lotaje y riesgo.
- Límite de pérdida y mecanismos de protección.
- Horarios y sesiones.
- Logging y códigos de error.
- Includes y componentes compartidos.
- Eliminación de dependencias no autorizadas.
- Generación o normalización de preset `.set`.

Regla sobre cambios manuales

La preparación debe ser automatizada y determinista cuando sea viable. Si una corrección manual es necesaria:

1. El código preparado pasa a ser fuente mantenida en Git.
2. El cambio se realiza mediante diff revisable.
3. Se documenta por qué no pudo expresarse en la receta.
4. El build referencia el commit exacto.

Nunca se editará un archivo en la carpeta de MT5 sin trasladar el cambio a la fuente autoritativa.

Gate G2

- Receta identificada por commit.
- Código preparado con hash.
- Dependencias enumeradas.
- Preset normalizado con hash.
- Revisión de seguridad y riesgo completada.

9. Stage 3 — Compilación

Entorno de compilación

La compilación se realizará en un workspace limpio mediante MetaEditor, registrando:

- versión y build;
- ubicación lógica del toolchain;
- arquitectura/target;
- dependencias;
- comando o mecanismo utilizado;
- fecha y duración;
- salida y código de retorno.

Procedimiento

1. Crear un workspace temporal limpio.
2. Materializar código, includes y preset desde fuentes verificadas.
3. Confirmar hashes de entrada.
4. Ejecutar MetaEditor de forma controlada.
5. Capturar stdout, stderr o log de compilación.
6. Verificar que se generó el `.ex5` esperado.
7. Calcular hash y tamaño del ejecutable.
8. Guardar ejecutable y log como artefactos inmutables.
9. Completar el build manifest.

Política de errores y warnings

- Cualquier error rechaza el build.
- La política inicial recomendada es cero warnings.
- Una excepción requiere aprobación registrada y regla explícita.
- No se aprobará un ejecutable si el log está ausente o no corresponde a sus entradas.

Gate G3

- Código de salida correcto.
- Cero errores.
- Warnings dentro de la política aprobada.
- `.ex5`, log y manifest registrados.
- Hashes de todas las entradas y outputs consistentes.

10. Build manifest

Cada construcción producirá un manifiesto equivalente a:

```yaml
schema_version: 1
build_id: bld_01...
strategy_version_id: stv_01...

sqx:
  version: "..."
  build: "..."
  export_profile: "..."
  raw_export_sha256: "..."

preparation:
  git_commit: "..."
  git_dirty: false
  recipe: "sqx-to-mt5-v1"
  prepared_source_sha256: "..."
  preset_sha256: "..."

compiler:
  name: MetaEditor
  version: "..."
  build: "..."
  target: MT5

result:
  status: succeeded
  errors: 0
  warnings: 0
  executable_sha256: "..."

artifacts:
  - role: sqx-raw-export
    uri: "storage://..."
    sha256: "..."
  - role: prepared-source
    uri: "storage://..."
    sha256: "..."
  - role: executable
    uri: "storage://..."
    sha256: "..."
  - role: compiler-log
    uri: "storage://..."
    sha256: "..."
```

11. Stage 4 — Validación técnica

Compilar correctamente es necesario, pero no suficiente.

Controles estáticos

- Magic Number único y trazable.
- Símbolo y timeframe esperados.
- Parámetros dentro de límites.
- Includes autorizados.
- DLL y accesos externos deshabilitados salvo autorización.
- Convenciones de logs y manejo de errores.
- Mecanismo de protección y parada cuando aplique.

Controles de ejecución

- Inicialización correcta en MT5.
- Ausencia de errores inmediatos en Journal/Experts.
- Smoke backtest corto.
- Apertura, modificación y cierre de órdenes según diseño.
- Respeto de lotaje, stops, horarios y límites.
- Persistencia segura frente a reinicio cuando corresponda.

Comparación SQX/MT5

Se realizará una prueba de equivalencia sobre un periodo y dataset controlados. No se exige igualdad ciega si los motores modelan el mercado de forma diferente, pero toda desviación material debe explicarse.

Se acordarán tolerancias para:

- número y secuencia de operaciones;
- dirección de entradas;
- timestamps dentro de una tolerancia;
- precios y costes;
- beneficio y drawdown;
- comportamiento de stops y salidas.

Gate G4

- Todos los controles obligatorios aprobados.
- Diferencias SQX/MT5 dentro de tolerancia o justificadas.
- Evidencias almacenadas.
- Revisión humana completada cuando corresponda.

12. Stage 5 — Aprobación y paquete

Un build aprobado se vuelve inmutable y apto para despliegue. El paquete lógico incluye:

- `.ex5`;
- preset efectivo;
- build manifest;
- log de compilación;
- informe de validación;
- hashes y referencias a fuentes.

La aprobación registra responsable, fecha, gates y excepciones. Modificar cualquier elemento exige crear otro `BotBuild`.

13. Stage 6 — Environment y Deployment

Esta etapa ocurre dentro de la VPS MT5 mediante el runner y bajo control del operador local. TradingLab prepara un `JobPackage`, pero no instala remotamente el bot ni accede al terminal.

Environment

Debe describir como mínimo:

- alias del entorno;
- propósito: desarrollo, demo, incubación u otro;
- versión/build de MT5;
- broker y servidor mediante alias;
- tipo de cuenta sin credenciales;
- moneda y apalancamiento cuando sean relevantes;
- zona horaria;
- símbolos, sufijos y especificaciones;
- máquina/VPS mediante identificador interno;
- capacidades permitidas.

Deployment

Debe registrar:

- `deployment_id`;
- `bot_build_id`;
- `environment_id`;
- hash del `.ex5` verificado antes de instalar;
- hash del preset;
- símbolo, timeframe y gráfico/slot;
- Magic Number;
- fecha, responsable y estado;
- procedimiento de rollback.

El deployment informado por el runner se considera pendiente de reconciliación hasta que TradingLab recibe y verifica su `EvidenceBundle`.

Gate G5

- Build en estado `approved`.
- Entorno autorizado y disponible.
- Hash del ejecutable coincidente.
- Preset y asignación de Magic Number validados.
- No hay otro deployment incompatible en el mismo slot.
- Rollback preparado.
- JobPackage válido y no expirado.
- Versión del runner compatible.
- Aprobación local registrada mediante acknowledgement.

14. Stage 7 — Incubación

La incubación se modela como un `Experiment` de tipo `incubation`. Cada periodo de ejecución es un `Run` asociado al deployment.

El runner administra localmente inicio, estado, parada y recolección. TradingLab recibe estados y resultados de forma asíncrona; la falta temporal de Drive no concede acceso remoto ni invalida automáticamente una ejecución ya aprobada.

Durante la incubación se registra

- inicio y fin;
- conectividad y uptime;
- versión del terminal;
- eventos de reinicio;
- órdenes y operaciones;
- errores y warnings;
- cambios autorizados;
- métricas operativas y financieras;
- artefactos de evidencia.

Cambios durante una incubación

No se reemplaza el `.ex5` ni el preset dentro del mismo run. Un cambio material requiere:

1. Cerrar o invalidar el run vigente.
2. Crear otro build o deployment según corresponda.
3. Iniciar un nuevo run.

Esto mantiene interpretable la evidencia.

Gate G6 — Cierre

- Artefactos y logs recopilados.
- Métricas importadas y validadas.
- Incidentes explicados.
- Decisión registrada: validar, rechazar, extender o iterar.
- Deployment detenido, mantenido o revertido explícitamente.
- EvidenceBundle devuelto y relacionado con el JobPackage.
- Nivel de verificación asignado tras la reconciliación.

15. Modelo de catálogo

`bot_builds`

Campos conceptuales:

- `id`
- `strategy_version_id`
- `status`
- `preparation_commit`
- `raw_export_artifact_id`
- `prepared_source_hash`
- `preset_hash`
- `compiler_name`

- `compiler_build`
- `executable_artifact_id`
- `compiler_log_artifact_id`
- `errors_count`
- `warnings_count`
- `approved_by`
- `approved_at`
- timestamps

`environments`

- `id`
- `name`
- `purpose`
- `mt5_build`
- `broker_alias`
- `server_alias`
- `account_alias`
- `timezone`
- `symbol_specification_artifact_id`
- `status`

`deployments`

- `id`
- `bot_build_id`
- `environment_id`
- `preset_artifact_id`
- `symbol`
- `timeframe`
- `magic_number`
- `status`
- `deployed_at`
- `stopped_at`
- `deployed_by`

El run de incubación referencia el `deployment_id`; no es necesario duplicar todos sus campos.

16. Ubicación en el repositorio

```
projects/<project-slug>/
├── configs/
│   ├── strategy/
│   ├── build/
│   └── incubation/
└── platforms/
    ├── sqx/
    │   ├── export-profiles/
    │   ├── templates/
    │   └── adapters/
    └── mt5/
        ├── src/
        ├── include/
        ├── presets/
        └── tests/

packages/
├── bot-pipeline/
└── exchange-contracts/

apps/
└── mt5-runner/
```

- `configs/build/` : recetas y políticas de construcción.
- `configs/incubation/` : configuración declarativa de incubación.
- `sqx/export-profiles/` : perfiles controlados de exportación.
- `sqx/adapters/` : transformaciones SQX a la representación preparada.
- `mt5/src/` : fuente MQL5 mantenida por el equipo.
- `mt5/include/` : componentes compartidos versionados.
- `mt5/presets/` : presets seguros y pequeños.
- `mt5/tests/` : fixtures y pruebas técnicas.
- `packages/bot-pipeline/` : lógica reutilizable de preparación, build, validación y empaquetado.
- `packages/exchange-contracts/` : esquemas y validadores de jobs, acknowledgements y evidencia.
- `apps/mt5-runner/` : aplicación local ejecutada en la VPS MT5.

Los originales de SQX, fuentes generadas, `.ex5` , logs e informes viven en storage cuando son outputs del pipeline. Git conserva las recetas, componentes propios, configuraciones declarativas y tests.

17. Fuentes de verdad

Elemento	Fuente autoritativa
Estrategia y configuración declarativa	Git
Exportación original SQX	Storage
Receta y componentes de preparación	Git
Código manual mantenido	Git
Código generado preparado	Storage + hash en build

Elemento	Fuente autoritativa
Estado y manifest del build	Catálogo
`.ex5` y log de compilación	Storage
Evidencias de validación	Storage + catálogo
Configuración efectiva de incubación	Git + hash en catálogo
Estado de deployment	Catálogo
Métricas de incubación	Catálogo
Código y tests del runner	Git `main`
RunnerRelease	Storage + manifiesto/hash
JobPackage	Storage + catálogo
Acknowledgement y EvidenceBundle	Storage + catálogo
Copias en Google Drive	Transporte temporal, nunca fuente de verdad
Decisión de aceptación/rechazo	Catálogo; ADR o bitácora si es relevante

18. Idempotencia y deduplicación

- Una combinación idéntica de entradas, receta y compilador no debe crear builds distintos accidentalmente.
- El hash del original detecta exportaciones ya registradas.
- El hash del ejecutable detecta binarios idénticos.
- Reintentar una compilación fallida puede crear un intento, pero no alterar su historial.
- Reimportar resultados de incubación utiliza claves de idempotencia y restricciones únicas.

19. Fallos, reintentos y rollback

Fallo de preparación

Se conserva el diagnóstico. La corrección genera una nueva revisión de receta o fuente.

Fallo de compilación

El build queda `failed` con su log. No se borra ni se recicla su identidad.

Fallo de validación

El build pasa a `rejected`. Una corrección material produce otro build.

Fallo de deployment

Se registra el incidente, se detiene el despliegue y se aplica el rollback previsto. No se inicia la incubación.

Fallo durante incubación

El run se cierra como `failed` o `invalidated` según la causa. Se preservan logs y estado para análisis.

20. Seguridad

- Nunca guardar credenciales de cuentas MT5 en Git, manifests o bitácoras.
- No permitir acceso remoto desde TradingLab a la VPS MT5.
- No instalar credenciales del monorepo principal en la VPS durante la fase inicial.
- No autoejecutar paquetes por aparecer en Google Drive.
- No aceptar shell o PowerShell arbitrario desde un JobPackage.
- Utilizar alias internos para cuentas, servidores y máquinas.
- Restringir DLL, red y filesystem por política.
- Verificar hashes antes de desplegar.
- Separar permisos para construir, aprobar y desplegar cuando el equipo lo permita.
- Registrar toda excepción de seguridad.
- Impedir que un agente despliegue o active bots sin la aprobación definida.
- Ejecutar el runner con usuario de privilegio mínimo y allowlist de acciones, rutas, símbolos y timeframes.

21. Observabilidad

Los logs del pipeline deben incluir:

- `correlation_id`;
- `project_id`;
- `strategy_version_id`;
- `build_id`;
- `deployment_id`;
- `run_id`;
- etapa y gate;
- versión de SQX, MetaEditor y MT5 cuando corresponda.

Se medirán duración y resultado de cada etapa, builds fallidos, warnings, deployments activos, integridad de artefactos y eventos de incubación.

22. Roles y aprobaciones

Una persona puede cubrir varios roles al inicio, pero las responsabilidades deben distinguirse:

- **Investigador:** selecciona candidato y define experimento.
- **Preparador:** mantiene recetas y adaptaciones.
- **Builder:** ejecuta la compilación controlada.
- **Reviewer:** revisa gates y excepciones.
- **Deployer:** instala el build aprobado.
- **Operador:** supervisa incubación y responde a incidentes.

Se acordará qué gates requieren aprobación humana obligatoria.

23. Interfaz futura

La CLI podrá ofrecer:

```
tradinglab bot prepare
tradinglab bot compile
tradinglab bot test
tradinglab bot approve
tradinglab bot package
tradinglab bot deploy
tradinglab incubation start
tradinglab incubation status
tradinglab incubation close
```

API y MCP utilizarán los mismos casos de uso. Ninguna interfaz podrá saltar estados o gates sin una excepción registrada.

24. Implementación incremental

Iteración 1 — Procedimiento asistido

- Definir manifests y checklist.
- Ejecutar preparación/compilación con scripts controlados.
- Registrar artefactos y hashes.
- Mantener aprobaciones humanas.

Iteración 2 — CLI reproducible

- Automatizar workspace, compilación y empaquetado.
- Persistir builds, gates y deployments.
- Añadir tests de regresión.

Iteración 3 — Incubación administrada

- Automatizar despliegue controlado y recolección de logs.
- Incorporar alertas y controles de integridad.
- Añadir workers si la concurrencia lo requiere.

Iteración 4 — Integración con agentes

- Crear skills para preparar, verificar y analizar.
- Exponer operaciones seguras mediante MCP.
- Mantener aprobación humana para despliegue y activación.

25. Criterios de aceptación del primer pipeline

1. La estrategia candidata y su versión están registradas.
2. El original de SQX puede recuperarse y verificarse.

3. La preparación puede repetirse o sus pasos manuales están documentados.
4. La versión/build de MetaEditor está identificada.
5. El `.ex5`, log y manifest tienen hashes consistentes.
6. Los gates muestran evidencia y resultado.
7. El build aprobado no puede modificarse.
8. El deployment verifica ejecutable, preset y entorno.
9. La incubación referencia un único deployment.
10. Un cambio material produce otro build, deployment o run.
11. Los fallos se conservan y pueden diagnosticarse.
12. No hay secretos en Git, catálogo, manifests o bitácora.

26. Decisiones pendientes

1. Builds de SQX, MetaEditor y MT5 admitidos.
2. Política definitiva de warnings.
3. Tolerancias de equivalencia SQX/MT5.
4. Controles de riesgo obligatorios.
5. Estrategia de Magic Numbers.
6. Primer Environment de incubación.
7. Responsables y gates con aprobación humana.
8. Política para componentes MQL5 compartidos.
9. Duración y criterios de éxito de incubación.
10. Condiciones futuras para promover a producción.
11. Runtime, permisos y política de actualización del MT5 Runner.
12. Acciones del runner que exigirán aprobación local.
13. Permisos y retención de las carpetas de Google Drive.
14. Momento de adopción de releases inmutables y attestations de GitHub.

27. Conclusión

El pipeline convierte una exportación de SQX en evidencia operativa controlada. La separación entre `StrategyVersion`, `BotBuild`, `Deployment` e incubación permite saber no solo qué estrategia se investigó, sino qué binario exacto se ejecutó, cómo se construyó y bajo qué condiciones produjo sus resultados.